

Study_Case1



4143108 - Software Architecture and Design

Nama: Yos Ivan Sirait

NIM: 11423025

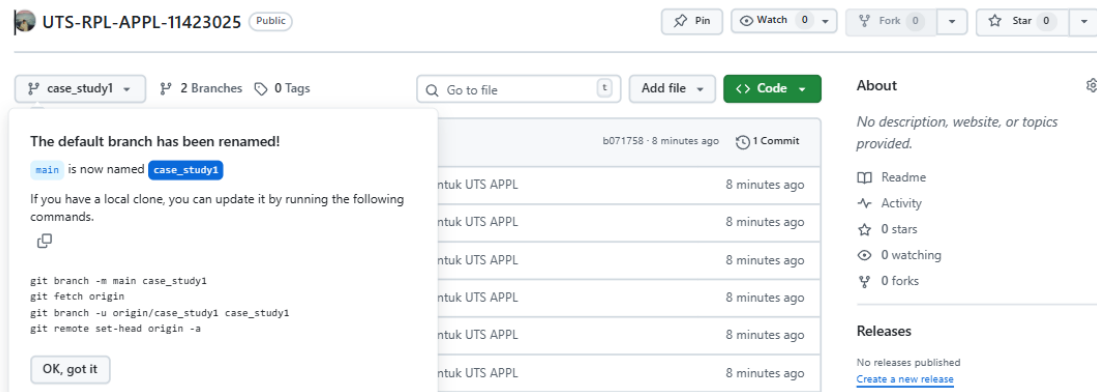
Sarjana Terapan Teknologi Rekayasa

Perangkat Lunak

FAKULTAS VOKASI

INSTITUT TEKNOLOGI DEL

Case Study 1 : "MediTrack - A Digital Healthcare Platform"



1. Arsitektur yang Dipilih

Berdasarkan kebutuhan platform dan ekspektasi pertumbuhan, saya memilih arsitektur Monolitik Modular menggunakan Next.js (App Router) sebagai framework full-stack dan Supabase sebagai Backend-as-a-Service (PostgreSQL + Auth + Storage). Alasan utama: kesederhanaan, kecepatan pengembangan, dan integrasi yang kuat dengan Supabase.

a. Justifikasi dengan Prinsip Arsitektur

1. Kesederhanaan (Simplicity)

- Satu codebase memudahkan pengembangan, debugging, dan deployment.
- Next.js menyediakan API routes dan serverless functions yang terintegrasi, sehingga tim dapat fokus pada fitur tanpa kompleksitas jaringan.

2. Kecepatan Pengembangan (Development Velocity)

- Perubahan lintas modul dapat dilakukan dengan cepat karena semua kode berada dalam satu proyek.
- Supabase mempercepat development dengan menyediakan autentikasi, realtime, dan storage siap pakai.

3. Konsistensi dan Integritas Data (Consistency)

- Menggunakan satu database (Supabase) memastikan transaksi ACID untuk operasi yang melibatkan banyak modul (misal booking appointment yang memerlukan update EHR dan pembayaran).
- Penting untuk data medis yang membutuhkan integritas tinggi.

b. Trade-offs dan Keterbatasan

- Skalabilitas horizontal terbatas – karena satu aplikasi, scaling harus dilakukan pada level instance (load balancing) bukan per modul
- Keterikatan modul – perubahan pada satu modul berpotensi mempengaruhi modul lain. Untuk mengurangi risiko, diterapkan modularisasi dalam struktur folder dan pemisahan tanggung jawab.
- Teknologi terikat – semua harus menggunakan stack yang sama (Next.js, Supabase), tidak bisa memilih teknologi berbeda per modul.

2. System Decomposition & Modeling

Modul	Tanggung Jawab	Entity Utama
IAM (Identity)	Registrasi, Login, dan RBAC (Role-Based Access Control)	User, Role
Appointment	Janji Temu	Appointment, doctor schedule
EHR	Menyimpan Riwayat medis	MedicalRecord
Pharmacy	Pengelolaan resep digital dan stok obat	Medication Stock
Payment	Payment Gateway	Payment

3. System Decomposition & Modeling

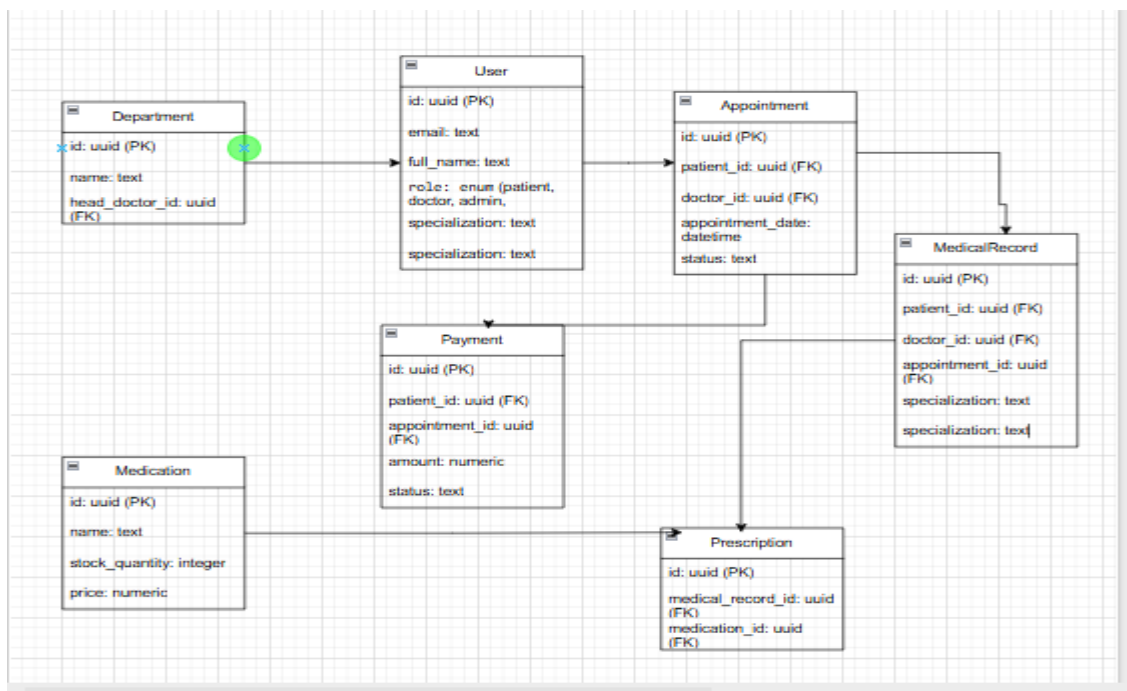
1. Break down the MediTrack system into core modules or services based on your chosen architecture.

```
meditrack/
├── app/
│   ├── api/
│   │   ├── auth/           # Authentication endpoints
│   │   ├── users/          # User management
│   │   ├── appointments/   # Appointment management
│   │   ├── medical-records/ # EHR system
│   │   ├── pharmacy/
│   │   │   ├── medications/ # Medication management
│   │   │   └── prescriptions/ # Prescription management
│   │   ├── payments/       # Payment processing
│   │   └── analytics/      # Analytics data
│   ├── auth/
│   │   ├── login/          # Login page
│   │   └── register/       # Registration page
│   ├── dashboard/
│   │   ├── patient/        # Patient dashboard
│   │   ├── doctor/         # Doctor dashboard
│   │   └── admin/          # Admin dashboard
│   ├── globals.css         # Global styles & theme
│   ├── layout.tsx          # Root layout
│   └── page.tsx             # Home page
├── components/
│   └── ui/                  # shadcn/ui components
├── lib/
│   ├── db.ts               # Database utilities
│   ├── auth.ts             # Authentication utilities
│   ├── api.ts              # API helpers
│   ├── validators.ts       # Zod validators
│   └── utils.ts            # General utilities
├── types/
│   └── index.ts             # TypeScript type definitions
├── scripts/
│   └── 01-init-schema.sql   # Database schema
├── package.json
├── tsconfig.json
└── tailwind.config.ts
```

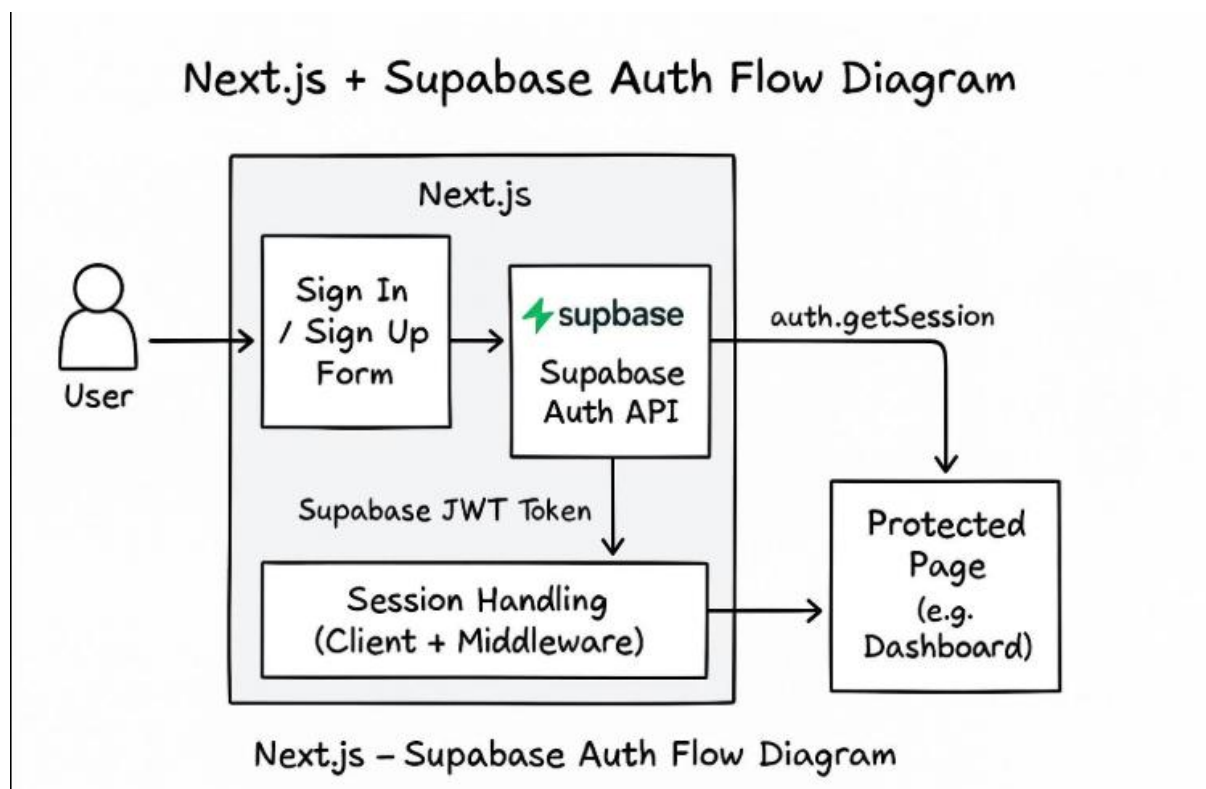
2. Relationships Diagram



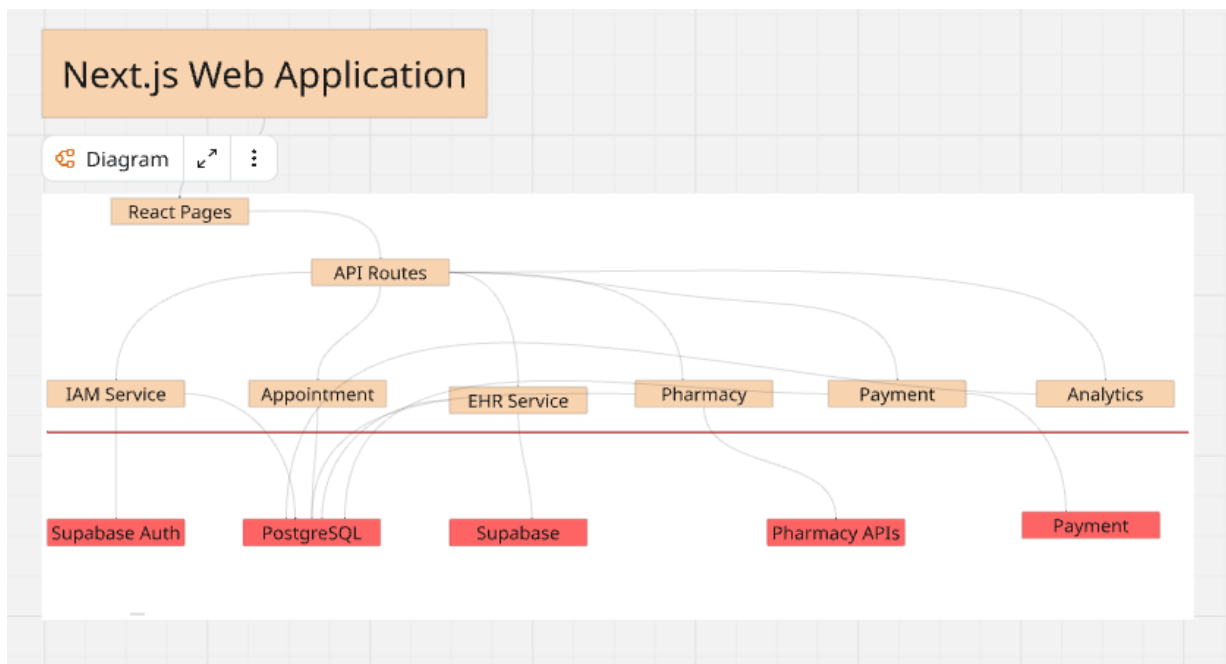
3. Use UML class diagrams or component diagrams to illustrate the design.



4. Architecture Visualization



Alur kerja sistem MediTrack dimulai ketika pengguna melakukan interaksi melalui Web Client berbasis Next.js, di mana permintaan (request) akan melewati lapisan Middleware untuk proses autentikasi dan pengecekan peran (RBAC) sebelum diteruskan ke API Routes yang terorganisir secara modular. Setiap rute API kemudian memanggil layanan bisnis yang spesifik—seperti Appointment Service atau EHR Service—yang bertugas mengolah logika data dengan berinteraksi langsung ke Supabase untuk autentikasi identitas dan penyimpanan data pada database PostgreSQL



Batasan Arsitektur dan Interaksi Sistem

1. Batasan Modular (Modular Boundaries) Dalam aplikasi Next.js ini, setiap modul (IAM, Appointment, EHR, dll.) dipisahkan secara logis ke dalam direktorinya masing-masing di bawah `lib/services/` atau `app/api/`. Hal ini memastikan bahwa logika bisnis terenkapsulasi di dalam modulnya sendiri. Komunikasi antar-modul ditangani melalui panggilan layanan internal atau antarmuka bersama (shared interfaces) yang terdefinisi dengan baik. Pendekatan ini mencegah akses database lintas-modul secara langsung sedapat mungkin untuk menjaga integritas modular.

2. Batasan Lapisan Layanan (Service Layer Boundaries) Lapisan layanan (Service Layer) bertindak sebagai pusat logika bisnis inti. Lapisan ini mengabstraksi akses data yang mendasari (Supabase/PostgreSQL) dan integrasi eksternal, sehingga menyediakan API yang bersih untuk dikonsumsi oleh frontend dan rute API (API routes). Pemisahan ini memastikan bahwa perubahan pada skema database atau penyedia layanan eksternal tidak berdampak langsung pada antarmuka pengguna (User Interface).

3. Aliran Data (Data Flow)

Berikut adalah urutan aliran data dalam sistem MediTrack:

1. Aksi Pengguna: Pengguna berinteraksi dengan Web Client (misalnya, melakukan pemesanan janji temu).

2. Permintaan (Request): Frontend mengirimkan permintaan ke rute API (API Route) yang sesuai.
3. Logika Bisnis: Rute API memanggil Layanan (Service) terkait (misalnya, Appointment Service).
4. Penyimpanan/Pengambilan Data: Layanan berinteraksi dengan Supabase Auth untuk identitas atau Database PostgreSQL untuk operasi data.
5. Integrasi Eksternal: Jika diperlukan, Layanan berkomunikasi dengan Sistem Eksternal (misalnya, memproses pembayaran melalui gateway).
6. Respons: Hasil dikirimkan kembali melalui Rute API ke Web Client, yang kemudian memperbarui tampilan antarmuka (UI).

5. Performance Bottlenecks & Mitigation Strategies

Sebagai sistem yang dipersiapkan untuk pertumbuhan cepat, beberapa potensi hambatan kinerja telah diidentifikasi beserta strategi mitigasinya:

- Database Connection Overload: Karena menggunakan arsitektur monolitik dengan banyak modul, beban pada satu database PostgreSQL bisa meningkat drastis.
 - ⇒ Mitigasi: Mengaktifkan Connection Pooling (menggunakan fitur Supabase/Prisma Accelerate) untuk x mengelola koneksi database secara efisien.